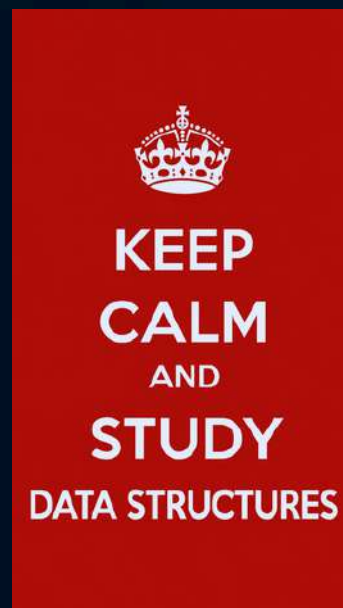




ESTRUCTURAS DE DATOS DE UN COMPILADOR



INTRODUCCIÓN AL ANÁLISIS
LÉXICO Y LENGUAJES FORMALES

ELIAB MOISES HERNANDEZ LOPEZ



Conceptos

Tokens, lexema, símbolos



Teoría

Árbol sintáctico, tabla de símbolos, traducción



Ejemplos

Instrucción DO, GetToken(), scanner



#1. Token

class Token:

- **Definición básica:** Es la estructura devuelta directamente por el scanner hacia el parser.
- **Campos básicos:** Cuenta con dos componentes principales:
 - **a) Código de token (número):** Indica el tipo o categoría del token (por ejemplo: Variable, INT-Const, Comma, Left-Parentheses).
 - **b) Hilera específica (lexema):** Muestra el valor particular o muestra específica leída (variables, constantes, etc.).



Estructuras

Token.py

ArbolSintactico.py

TablaSimbolos.py

TablaLiterales.py

VersionAbstracta.py

TraduccionDirigidaPorSintaxis.py

GetToken.py

CodigoIntermedio.py

ArchivosTemporales.py

Análisis Lexico I

AnálisisLexico.py

GetTokenEnEjecucion.py

ScannerEspacios.py

InstruccionDOFortran.py

ScannerDOFortran.py

GeneradoresScanner.py

DeteccionTokens.py

Análisis Lexico II

AnálisisLexicoFormal.py

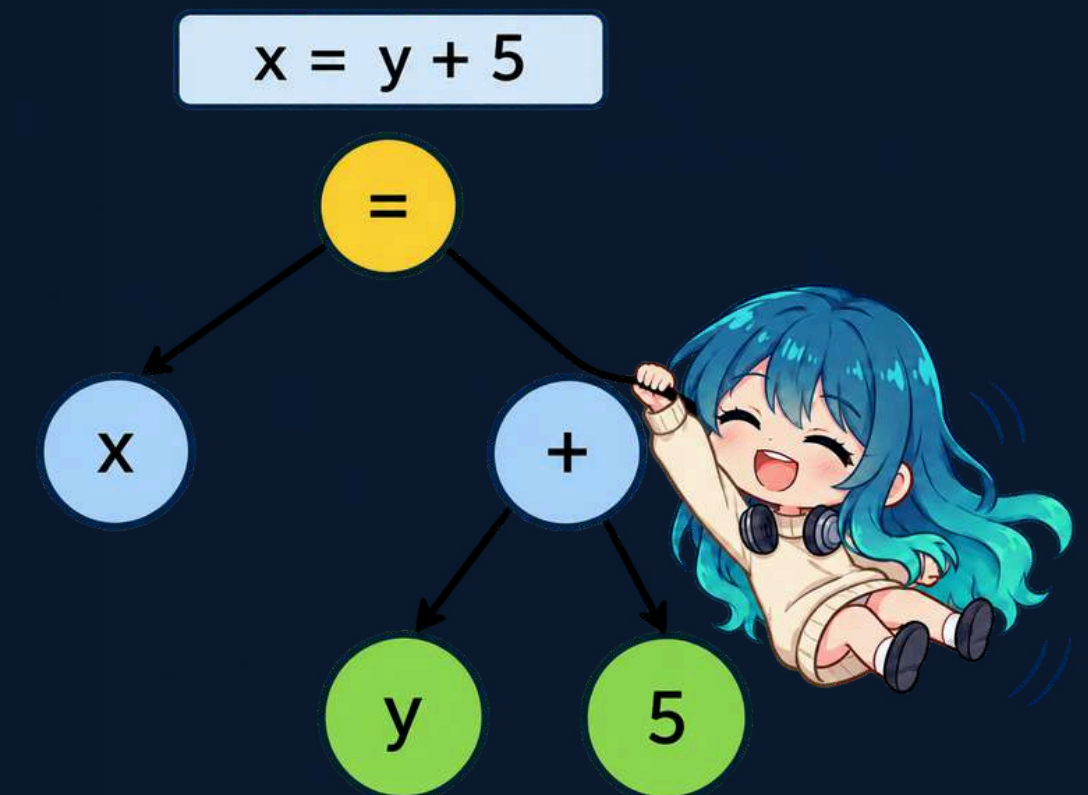
LenguajesFormales.py

Simbolo.py

#2. Árbol sintáctico

class ÁrbolSintáctico:

- **Origen y enriquecimiento:** Es creado por el parser y posteriormente decorado por el analizador semántico.
- **Representación:** Reside en memoria dinámica y existe una variable (puntero) que señala directamente a su raíz.
- **Estructura interna:** Sus campos son llenados por el parser y analizados por la fase semántica; sus nodos pueden contener punteros a otras estructuras.
- **Observación:** En algunos diseños de compiladores no existe explícitamente.



Estructuras

Token.py
ArbolSintactico.py
TablaSimbolos.py
TablaLiterales.py
VersionAbstracta.py
TraduccionDirigidaPorSintaxis.py
GetToken.py
CodigoIntermedio.py
ArchivosTemporales.py

Análisis Lexico I

AnalisisLexico.py
GetTokenEnEjecucion.py
ScannerEspacios.py
InstruccionDOFortran.py
ScannerDOFortran.py
GeneradoresScanner.py
DeteccionTokens.py

Análisis Lexico II

AnalisisLexicoFormal.py
LenguajesFormales.py
Simbolo.py

#3. Tabla de símbolos



Tabla de símbolos

Identificador	Tipo	Valor
x	int	10
y	float	3.5
total	int	42

class TablaDeSímbolos:

- **Información almacenada:** Conserva información asociada a todos los identificadores del programa (variables, procedimientos, funciones, constantes y tipos de datos).
- **Interacción:** Es la estructura de datos más utilizada durante la compilación e interactúa con el scanner, parser, analizador semántico y generadores de código.
- **Estructura física:** Puede estar formada por varias tablas asociadas y utiliza esquemas de acceso por hash para máxima eficiencia.

✓ Estructuras

Token.py
ArbolSintactico.py
TablaSimbolos.py
TablaLiterales.py
VersionAbstracta.py
TraduccionDirigidaPorSintaxis.py
GetToken.py
CodigoIntermedio.py
ArchivosTemporales.py

✓ Analisis Lexico I

AnalisisLexico.py
GetTokenEnEjecucion.py
ScannerEspacios.py
InstruccionDOFortran.py
ScannerDOFortran.py
GeneradoresScanner.py
DeteccionTokens.py

✓ Analisis Lexico II

AnalisisLexicoFormal.py
LenguajesFormales.py
Simbolo.py

#4. Tabla de literales

class TablaDeLiterales:

- **Contenido:** Almacena todas las constantes del programa (números e hileras/cadenas de caracteres).
- **Permanencia:** Su valor no cambiará durante la ejecución y deben residir en la memoria.
- **Punteros y optimización:** Puede recibir punteros desde la Tabla de Símbolos y ayuda a reducir el tamaño final del código generado.



▼ Estructuras

Token.py
ArbolSintactico.py
TablaSimbolos.py
TablaLiterales.py
VersionAbstracta.py
TraduccionDirigidaPorSintaxis.py
GetToken.py
CodigoIntermedio.py
ArchivosTemporales.py

▼ Analisis Lexico I

AnalisisLexico.py
GetTokenEnEjecucion.py
ScannerEspacios.py
InstruccionDOFortran.py
ScannerDOFortran.py
GeneradoresScanner.py
DeteccionTokens.py

▼ Analisis Lexico II

AnalisisLexicoFormal.py
LenguajesFormales.py
Simbolo.py

#5. Código intermedio

class CódigoIntermedio:

- **Posición en arquitectura:** Se ubica entre el Frontend y el Backend.
- **Formas de representación:** Puede estructurarse como un arreglo de punteros a hileras, un archivo de texto o una lista enlazada de hileras.
- **Optimización:** Permite reorganizar las instrucciones intermedias mediante eliminación, reemplazo o reordenamiento.



✓ Estructuras

Token.py
ArbolSintactico.py
TablaSimbolos.py
TablaLiterales.py
VersionAbstracta.py
TraduccionDirigidaPorSintaxis.py
GetToken.py
CodigoIntermedio.py
ArchivosTemporales.py

✓ Analisis Lexico I

AnalisisLexico.py
GetTokenEnEjecucion.py
ScannerEspacios.py
InstruccionDOFortran.py
ScannerDOFortran.py
GeneradoresScanner.py
DeteccionTokens.py

✓ Analisis Lexico II

AnalisisLexicoFormal.py
LenguajesFormales.py
Simbolo.py

#6. Archivos temporales



class ArchivosTemporales:

- **Ciclo de vida:** Son archivos de vida muy corta (de unos pocos segundos) que cumplen el ciclo:



- **Utilidad:** Enfoque típico en muchos compiladores para resolver problemas complejos de memoria y almacenamiento de manera simple.

EXPLORADOR

✓ Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

✓ Analisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

✓ Analisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

AnalisisLexico.py X

Análisis Léxico I

Conceptos generales y funcionamiento del scanner

```
1 begin
2   read x;
3   y := x + 10;
4   write y;
5 end
```



BEGIN	begin
READ	read
ID	x
SEMICOLON	;
ID	y
ASSIGNOP	:=
ID	x
PLUSOP	+
INTLITERAL	10
ID	y
SEMICOLON	;
END	end

Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Analisis Lexico I

- AnalisisLexico.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

Analisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#1. Análisis léxico

class AnálisisLéxico:

- **Concepto general:**
Corresponde al vocabulario del lenguaje.
- **Componente:** Ejecutado por el scanner, el cual toma la entrada fuente y la descompone en categorías léxicas mínimas denominadas tokens (equivalentes a las palabras en lenguaje natural).
- **Prioridad:** Es la primera fase que se aprende en la teoría de compiladores.

Ejemplo de análisis léxico:



class VersionAbstracta:



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

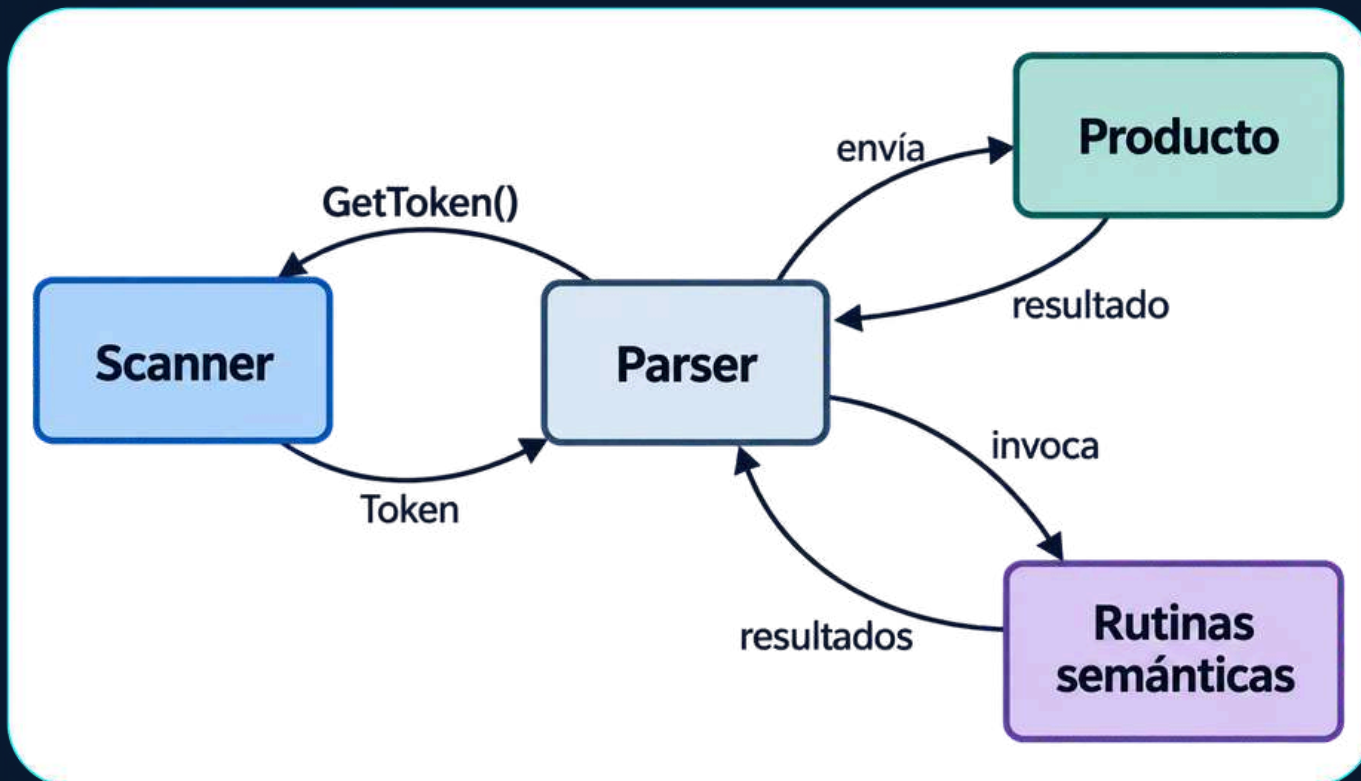
Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#2. Traducción dirigida por sintaxis

class TraducciónDirigidaPorSintaxis:

- **Flujo real:** Es la organización más usual en compiladores, donde el parser dirige todo el proceso y actúa como cliente del scanner.
- **Mecanismo:** El parser invoca al scanner mediante `GetToken()` cada vez que necesita un token e invoca rutinas semánticas en los puntos apropiados.
- **Alcance:** Su trabajo puede concluir al generar código intermedio o continuar hasta generar código en lenguaje máquina.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py**
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#3. GetToken()

class GetToken():

- **Definición:** Función que regresa el siguiente token del código fuente.
- **Representación:** Un token representa a un conjunto de hileras equivalentes sintácticamente.
- **Variedad:** Puede representar un solo elemento (ej. Left-parentheses) o un grupo reducido de elementos (ej. int, float, char).
- **Dominio:** Abarca un conjunto potencialmente infinito de elementos (números, hileras, etc.).

GetToken()

Programa fuente

```
int total = x + 42 ;
```

posición actual

Scanner

GetToken()

regresa el siguiente token

Token devuelto

Tipo: ID
Lexema: total

salta espacios

#4. Token (Estructura de 2 campos)

class TipoDeToken:

- **Tipo de token:** Categoría léxica (ej. variable, int, parentheses, comma).

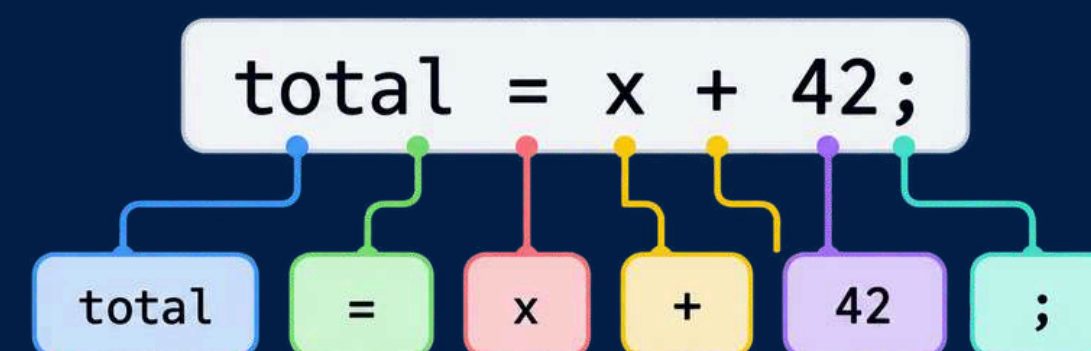
Tipo de token



class Lexema:

- **Puntero a la hilera particular (lexema):** Apunta al texto real en memoria

Lexema



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

Análisis Lexico II

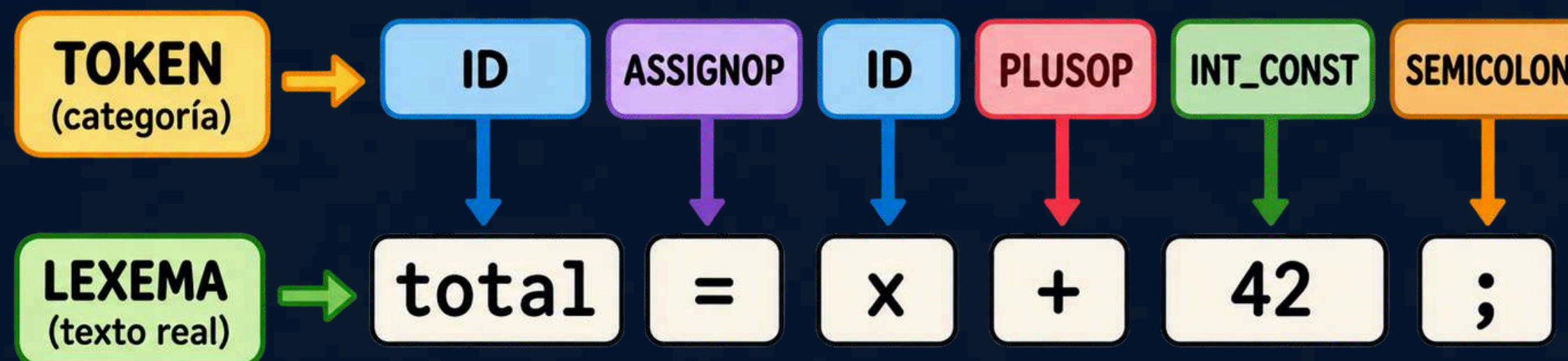
- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#5. Token vs. Lexema



class TokenvsLexema:

- **Relación:** Existe una relación 1 a n entre ambos.
- **Token:** Es la categoría léxica abstracta.
- **Lexema:** Es la hilera o secuencia concreta de caracteres en el programa que corresponde al token reportado por el scanner.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- DeteccionTokens.py

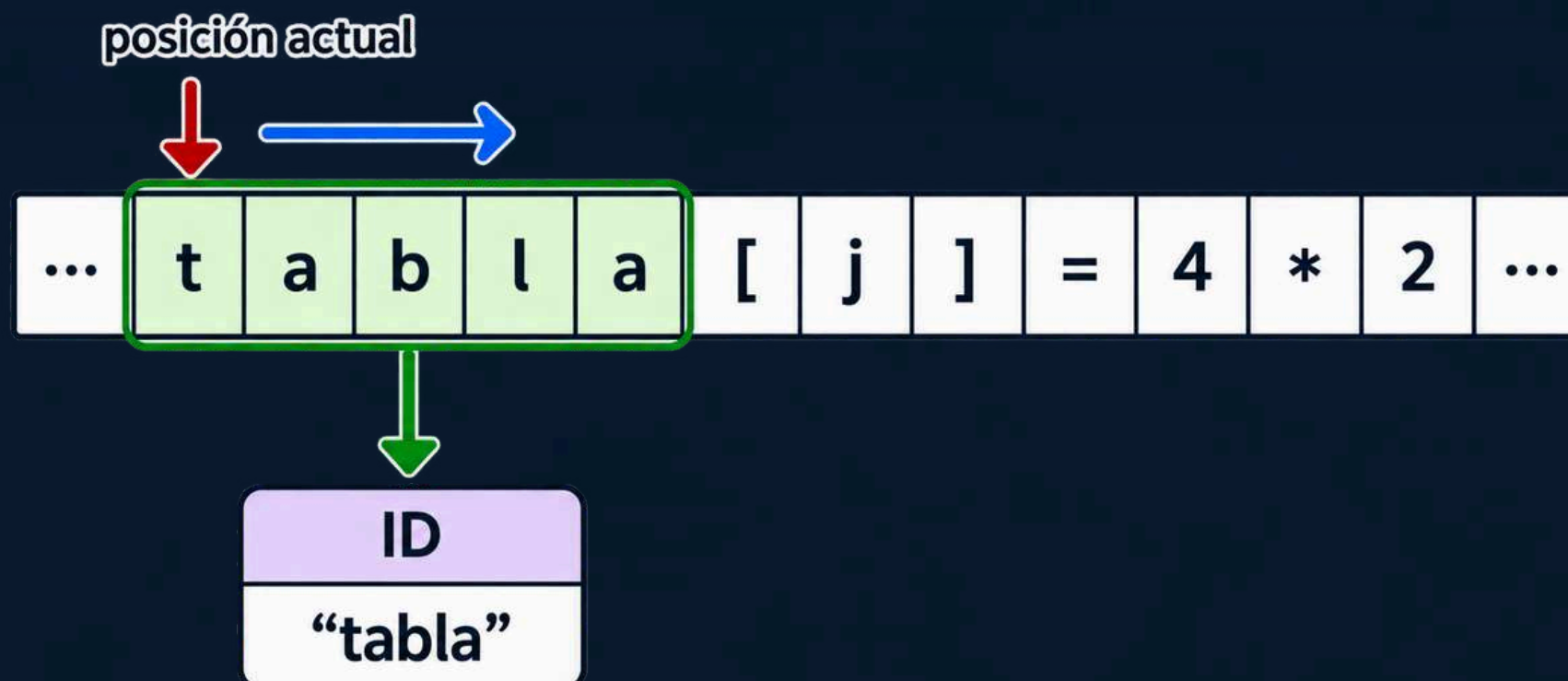
Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#6. GetToken() en ejecución

class GetToken()EnEjecución:

- **Estructura en memoria:** El programa completo reside en memoria como una hilera contigua de caracteres.
- **Procesamiento:** El scanner mantiene una posición actual (puntero), se salta los espacios (blancos, saltos de línea, pestañas, comentarios) y avanza hasta reconocer y regresar un token completo.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

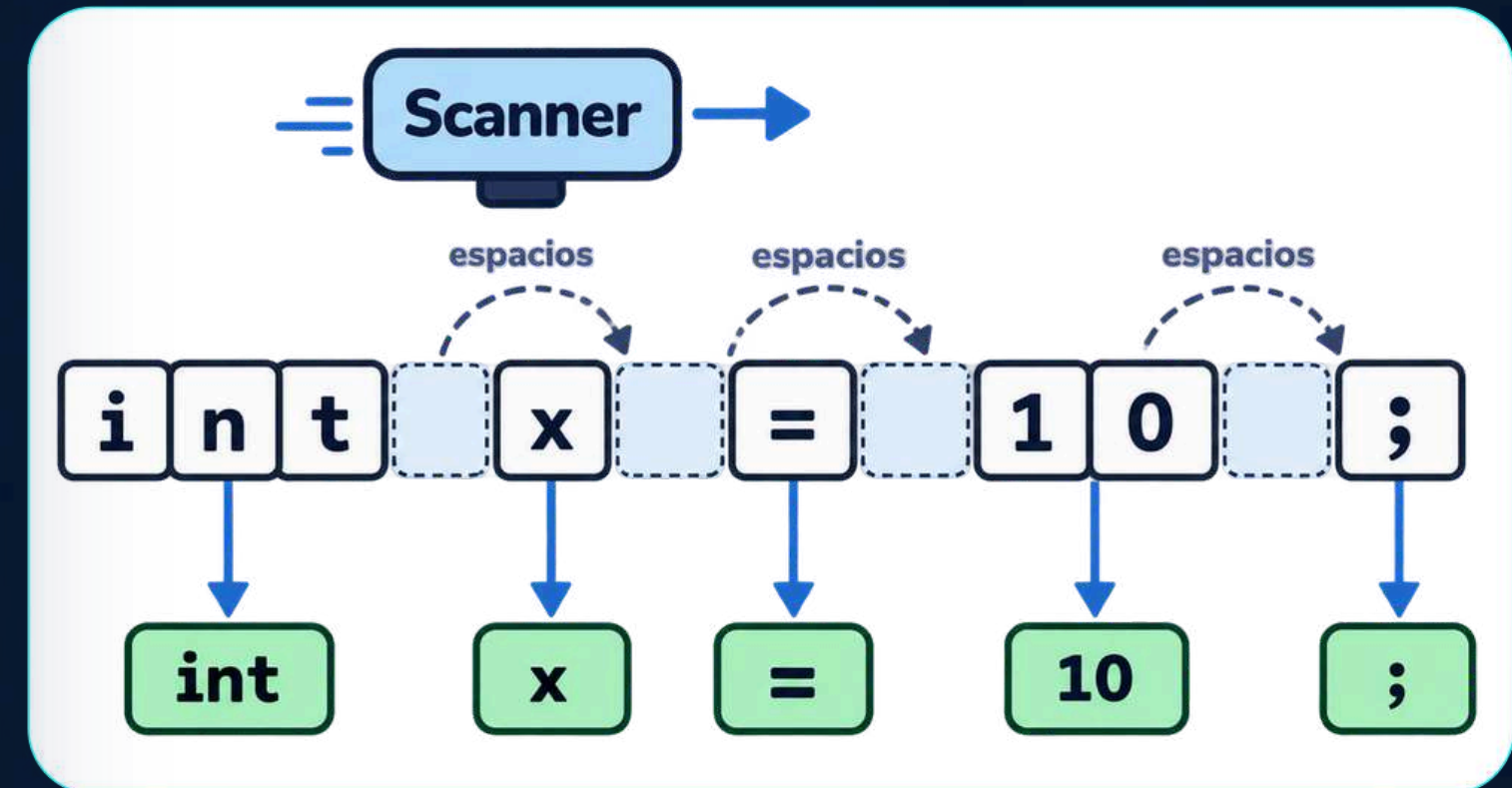
Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#7. Scanner y “espacios”

class ScannerYEspacios:

- **Regla general:** Usualmente no existe un token explícito para representar el "espacio", por lo que el scanner debe ignorarlos.
- **Antecedente histórico:** Surgió la idea de eliminar todos los blancos antes de compilar (enfoque de Backus en FORTRAN).



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

Análisis Lexico II

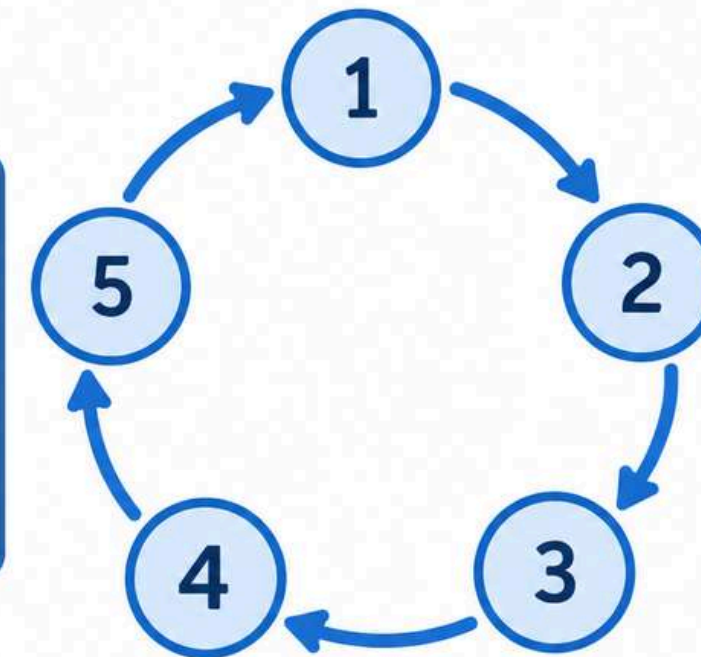
- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#8. Instrucción DO en FORTRAN

class InstrucciónDOFORTRAN:

- **Propósito:** Permite construir bucles iterativos sobre una secuencia de instrucciones, modificando una variable desde un valor inicial a uno final con un incremento en cada paso.

```
DO 10 I = 1, 5  
    SUM = SUM + I  
10 CONTINUE
```



Analisis Lexico I

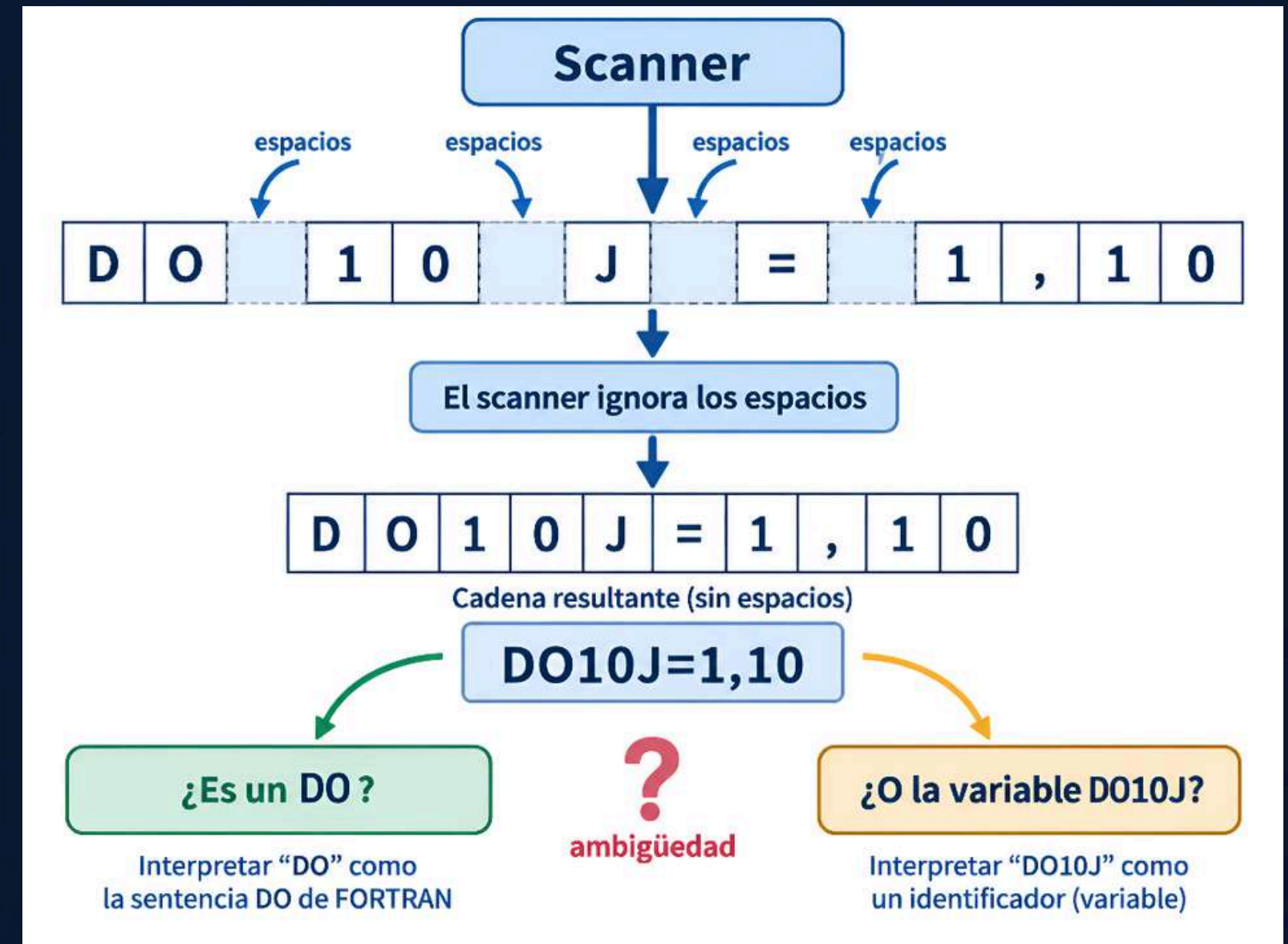
- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

Problema de eliminar blancos

- **Dilema del Scanner:** Al leer carácter a carácter, el scanner no sabe si se trata del inicio del ciclo DO o de la variable DO10J.

- **Ambigüedad y Backtracking:**
La ambigüedad se mantiene aún avanzando en la entrada; solo al llegar a la coma , (o al punto), el scanner descubre que era un bucle DO, debiendo retroceder (backtracking) para reconvertir la secuencia en 6 tokens individuales.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#10. Scanner y espacios (Importancia y función)

class ScannerYEspacios:

- **Conclusión:** Aunque los espacios no sean tokens, son esenciales como separadores.
- **Regla de oro:** "Los espacios son amigos del compilador"; eliminarlos completamente fue una mala idea debido a las ambigüedades que introduce.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

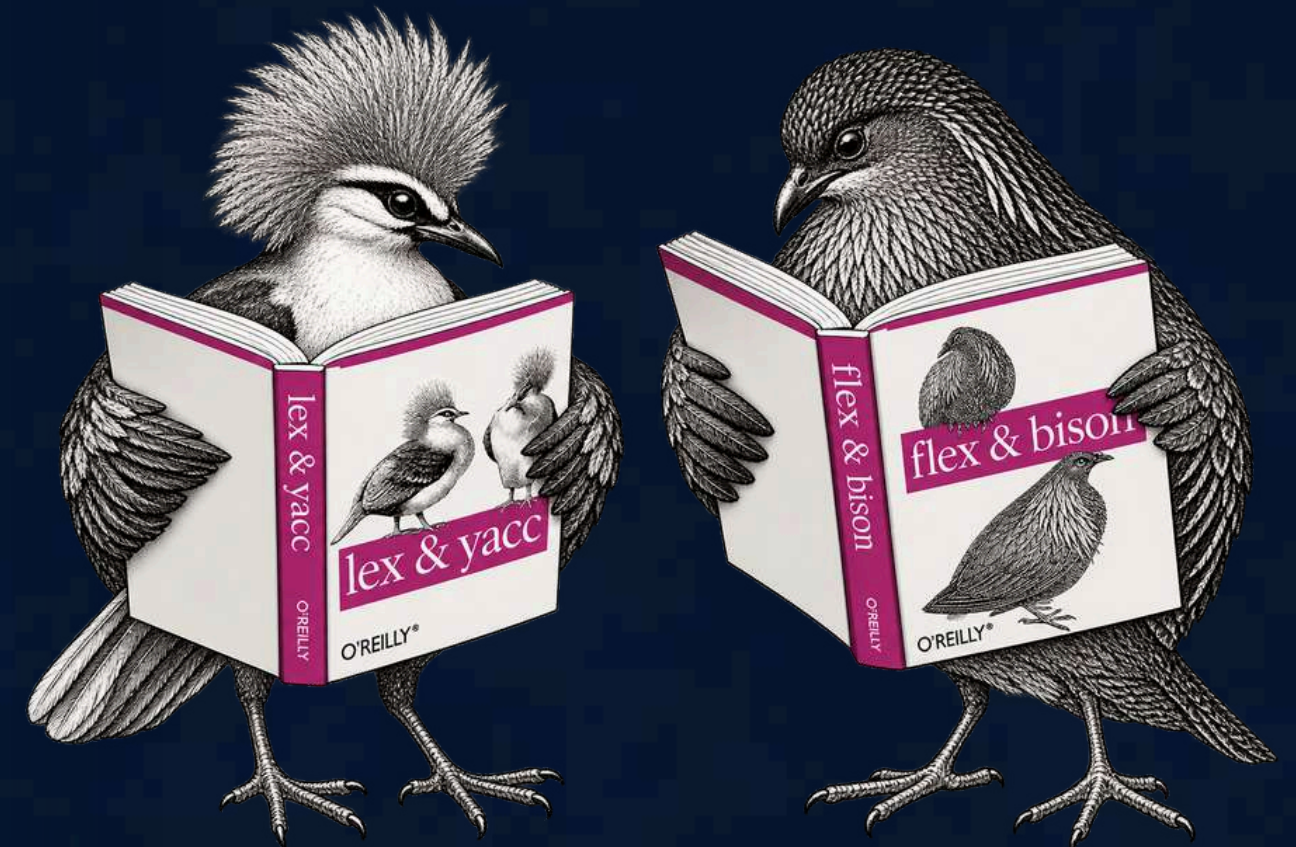
Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#11. Generadores de scanner

class GeneradoresDeScanner:

- **Automatización:** La mayor parte del análisis léxico se puede automatizar gracias a la teoría acumulada.
- **Herramientas:** Destacan Lex y Flex (reflejados en la literatura clásica Lex y Yacc o Flex y Bison).
- **Facilidad:** Generar un scanner es relativamente fácil, aunque siempre es posible o necesario agregar algo de código a mano.



Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

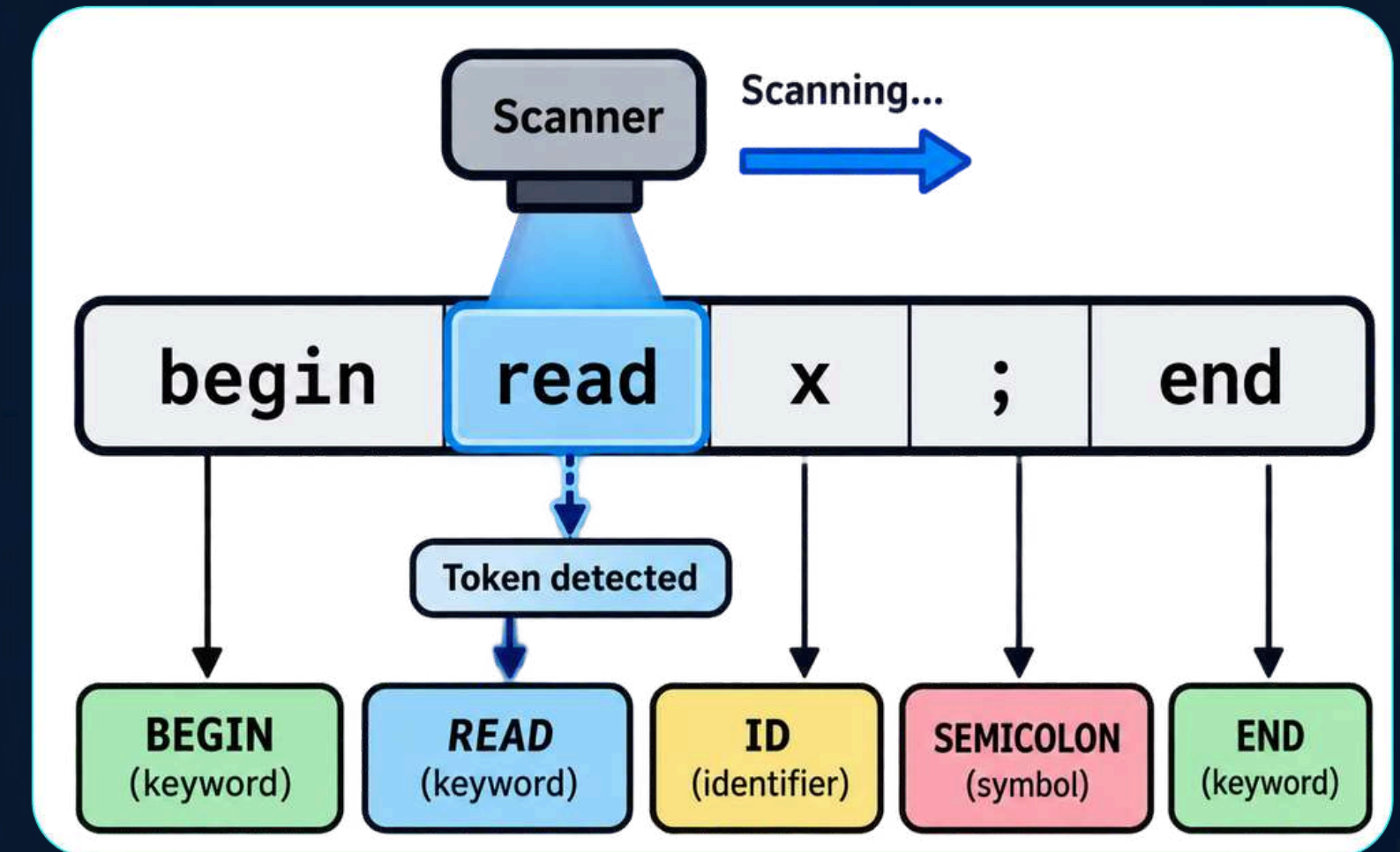
Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#12. Detección de tokens

class DetecciónDeTokens:

- **Función principal:** Avanzar en la entrada fuente hasta reconocer el token más grande posible (maximal munch).
- **Fundamento teórico:** Aplica técnicas derivadas de Lenguajes Formales, Teoría de Autómatas y Autómatas Determinísticos de Estados finitos.



Somos automatas
determinísticos de estados finitos



EXPLORER

▼ Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

▼ Analisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py
- DeteccionTokens.py

▼ Analisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

AnalisisLexicoFormal.py ×

Σ^*

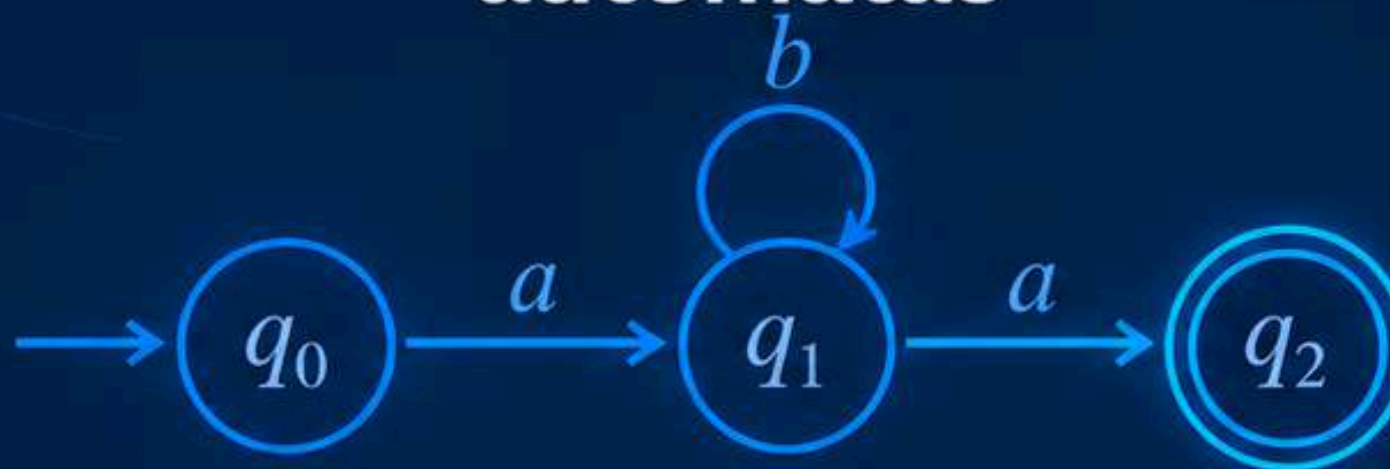


$L(M)$

Análisis Léxico II

Enfoque lenguajes formales y autómatas

$(a \mid b)^*$



$\delta(q, a)$

Estructuras

Token.py
ArbolSintactico.py
TablaSimbolos.py
TablaLiterales.py
VersionAbstracta.py
TraduccionDirigidaPorSintaxis.py
GetToken.py
CodigoIntermedio.py
ArchivosTemporales.py

Análisis Lexico I

AnalisisLexico.py
TokenVsLexema.py
GetTokenEnEjecucion.py
ScannerEspacios.py
InstruccionDOFortran.py
ScannerDOFortran.py
GeneradorDeScanner.py

Análisis Lexico II

AnalisisLexicoFormal.py
LenguajesFormales.py
Simbolo.py

#1. Lenguajes formales



class LenguajesFormales

- **Definición lingüística:** Habilidad humana para adquirir y usar complejos sistemas de comunicación o conjunto de sonidos articulados.
- **Categorías:** Lenguaje Natural, Lenguaje de Computadora, Lenguaje Matemático y Lenguaje Formal.

Estructuras

- Token.py
- ArbolSintactico.py
- TablaSimbolos.py
- TablaLiterales.py
- VersionAbstracta.py
- TraduccionDirigidaPorSintaxis.py
- GetToken.py
- CodigoIntermedio.py
- ArchivosTemporales.py

Análisis Lexico I

- AnalisisLexico.py
- TokenVsLexema.py
- GetTokenEnEjecucion.py
- ScannerEspacios.py
- InstruccionDOFortran.py
- ScannerDOFortran.py
- GeneradoresScanner.py

Análisis Lexico II

- AnalisisLexicoFormal.py
- LenguajesFormales.py
- Simbolo.py

#2. Símbolos

class Símbolos:

- **Naturaleza:** Es un concepto primitivo (con definición circular) que representa una idea, concepto o entidad con significado convencional.
- **Diferencia:** No es necesariamente lo mismo que un solo carácter y suele denotarse con letras como (a, b, c).



GRACIAS

CONTINUARÁ...

